LOGIN: A Large Language Model Consulted Graph Neural Network Training Framework

Chloe Kim
February 3, 2026
WSDM ‘25

Motivation

- Designing GNN variants is labour intensive
  - Requires extensive trial and error with both theoretical understanding and practical insights
- GNNs struggle with diverse graph types
  - Classic GNNs work well on homophilic graphs but fail on heterophilic scenarios where dissimilar nodes connect
- LLMs offer powerful semantic understanding
  - Rich open-world knowledge and emerging logical reasoning capabilities can enhance graph learning

Question

How can we leverage LLMs to optimize classic GNNs across various graph types/scenarios?

Existing LLM & Graph Integration Paradigms

1. LLMs-as-Predictors
   a. Directly use LLMs for predictions
   b. High computational cost
   c. Ignores existing GNN advantages -> results in only moderate performance or substantial computing resource consumption
2. LLMs-as-Enhancers
   a. Preprocess data before GNN
   b. One-time enhancement only
   c. No interactive integration

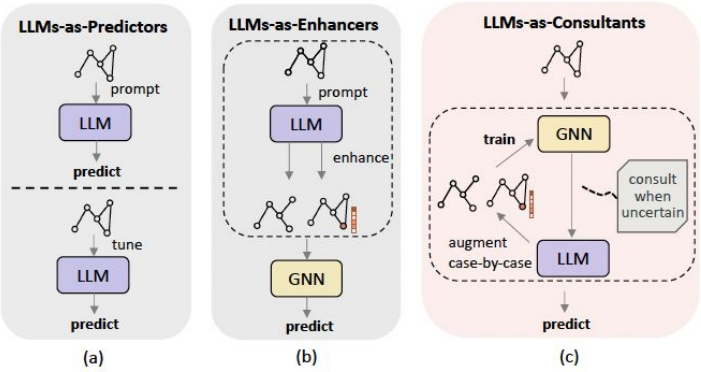


Figure 1: The paradigms for integrating LLMs with graphs.

Proposed Approach: LLMs-as-Consultants

- Interactive integration of LLMs within GNN training process

LLM-as-Consultants

- Consults LLMs on uncertain nodes during GNN training (integrate LLMs directly into GNN training)
  - Enhances GNN performance with acceptable additional resource usage

1. Identify uncertain nodes
   a. Use GNN predictive uncertainty
2. Craft concise prompts
   a. Include semantic + topological information
3. Utilize LLM responeses
   a. Complementary mechanism for correct & wrong predictions

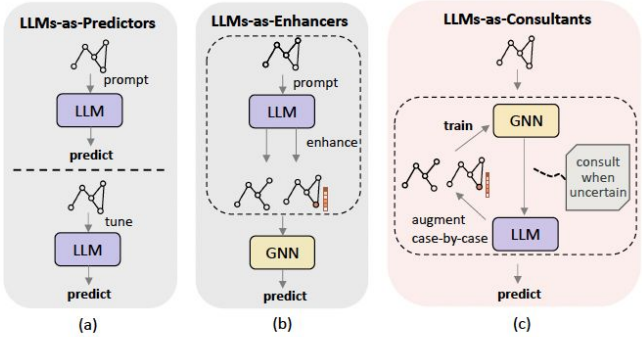


Figure 1: The paradigms for integrating LLMs with graphs.

LOGIN (**L**LM c**O**nsulted **G**NN tra**IN**ing) Framework

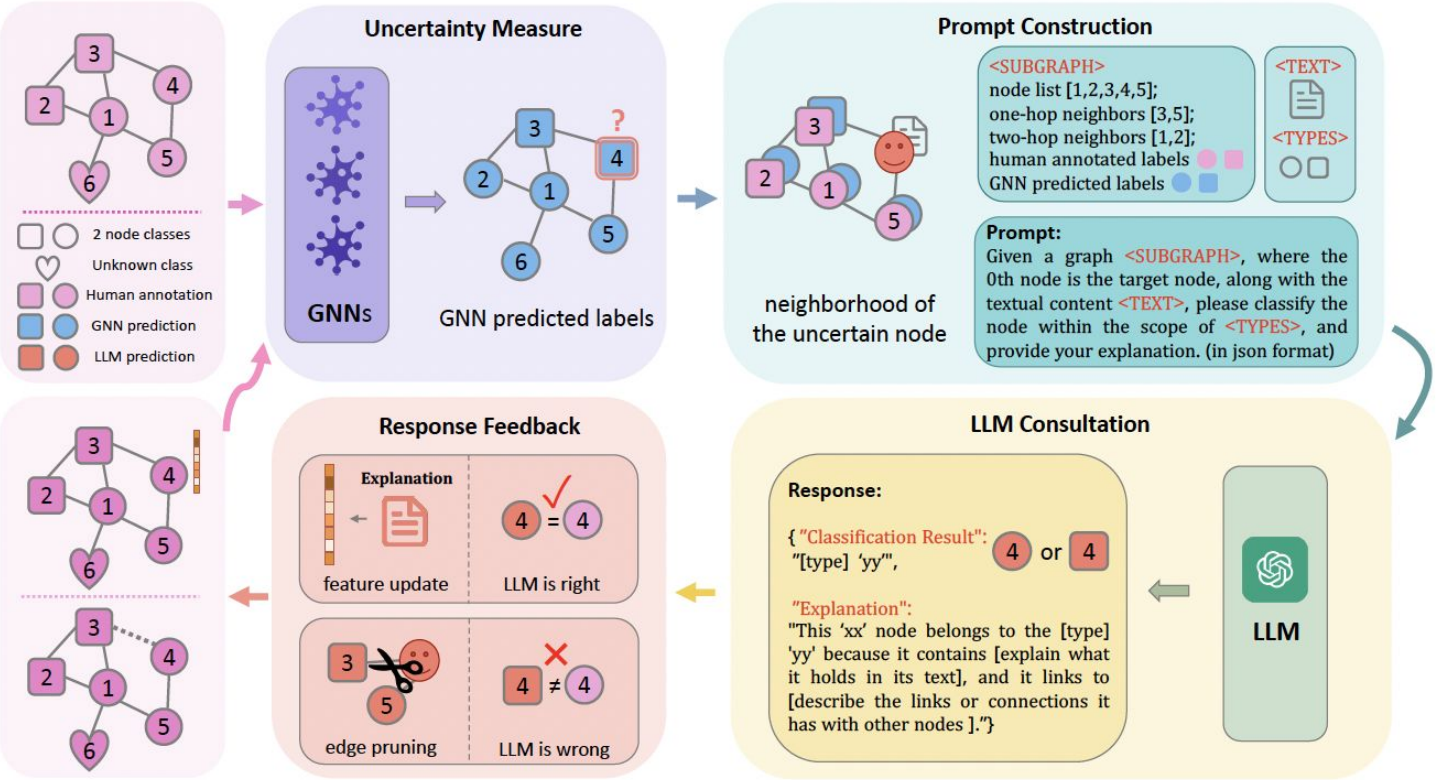
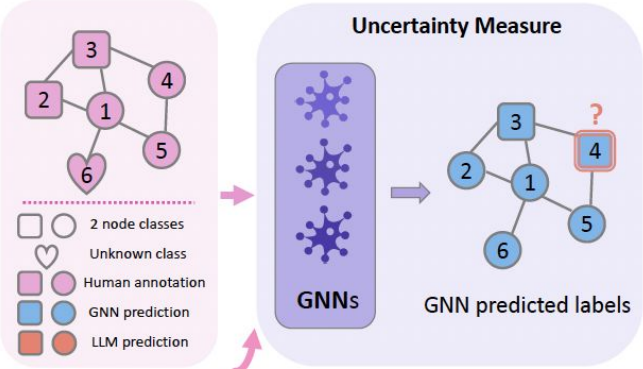


Figure 2: The pipeline of LOGIN.

Node Selection via Uncertainty

- Consult LLMs *only* with “partial difficult” nodes & adopt variance of predictions from several GNNs as a surrogate indicator
- All nodes:
  - Underutilizes GNN capacity
  - GNNs handle simple nodes well
  - LLM interaction is slow
- Focus on difficult nodes only

- Solution: Monte Carlo Dropout
  - Measures prediction uncertainty
  - Outputs a vector of model uncertainty score of each node

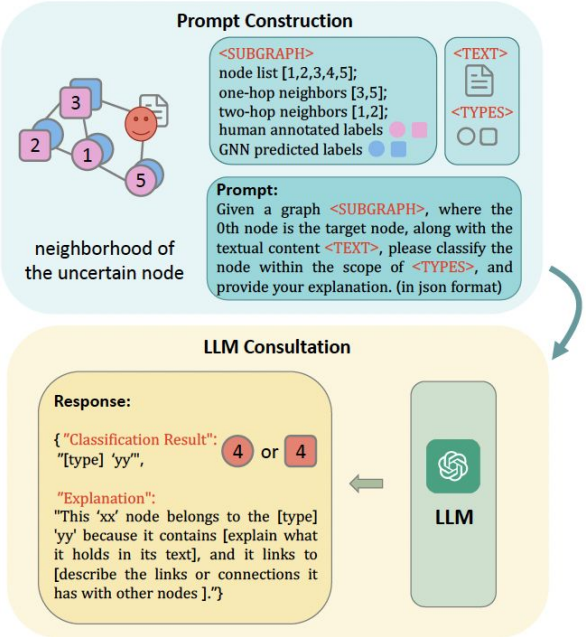


LLM Consultation - Prompt Construction

Zero-shot, node-by-node consultation with three elements:

1. Instruction
   a. Node classification task explanation, context of graph type, category spectrum
2. Input Data
   a. Node text, one-hop & two-hop neighbours, neighbour labels

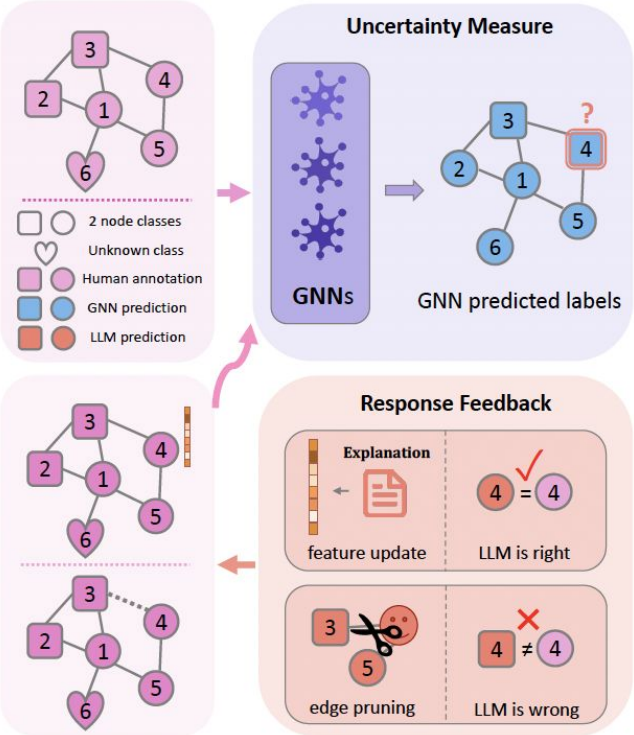
      (human annotated and GNN predictions)
3. Output Indicator
   a. JSON with explanation



Response Feedback

- LLM predicts correctly:
  - Prediction = ground truth label
  - Feature update
    - Append explanation from LLM to original text
    - Attain new node embedding by encoding new text
      - Semantic enhancement
- LLM predicts wrong:
  - Prediction ≠ ground truth label
  - Structure refinement (coping mechanism)
    - Prune edges below cosine similarity threshold
    - Denoise topology around wrong node

- Both paths contribute to improve GNN retraining
  - Semantic upgrade or structure (topology) refinement
  - “fault-tolerant but also fault-utilized”



LOGIN (**L**LM c**O**nsulted **G**NN tra**IN**ing) Framework

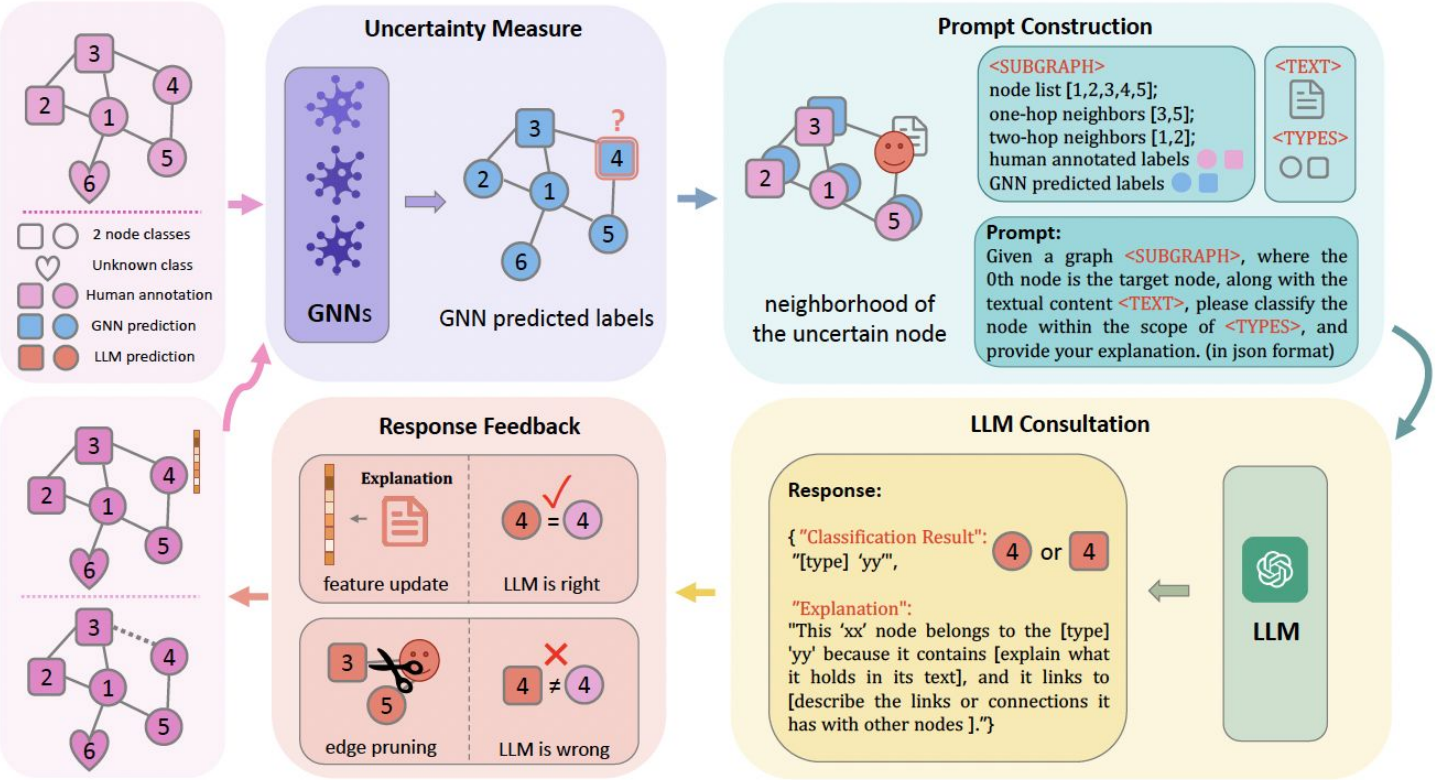


Figure 2: The pipeline of LOGIN.

Training and Testing Process

- Training phase
  a. Pre-train GNN
     - Initial node classification
  b. Identify uncertain nodes
     - Via MC dropout variance
  c. Consult LMM
     - Get predictions and explanations
  d. Apply feedback
     - Update features OR prune edges
  e. Retrain GNN
     - On augmented graph

- Testing Phase
  - No LLM consultation
    - Use trained GNN directly
    - Fast inference
    - No additional cost
  - The GNN learned from:
    - Enhanced node features
    - Refined graph structure
    - LLM semantic knowledge

Experimental Setup

- Datasets (6 TAGs)
  - Homophilic: Cora, PubMed, Arxiv-23
  - Heterophilic: Wisconsin, Texas, Cornell
- Methods Compared
  - LLM: Vicuna-v1.5-7b
  - Classic GNNs: GCN, GraphSAGE, MixHop
  - Advanced GNNs: JK-Net, H2GCN, APPNP, GCNII, SGC, SSP
- LOGIN Variants Tested
  - OR: Backbone model trained on original shallow features
  - FT: node embeddings from fine-tuned LM
  - LO: model trained with LOGIN framework
  - Metric is test set accuracy (mean accuracy and standard error from five runs with varied random data splits)

Main Results / Performance Comparison (RQ1)

RQ1: Does the LOGIN framework achieve performance comparable to state-of-the-art GNNs?

Table 1: Performance comparison for node classification on homophilic and heterophilic graphs. The best results and the second best results among the classic GNNs with and without LOGIN and the advanced models are bold and underlined respectively.

Method			Cora	PubMed	Arxiv-23	Wisconsin	Texas	Cornell
Classic GNNs	MLP	OR	0.6438 ± 0.0331	0.8805 ± 0.0032	0.6759 ± 0.0027	0.8113 ± 0.0718	0.8105 ± 0.0730	0.7538 ± 0.0669
		FT	0.6897 ± 0.0102	0.9486 ± 0.0030	0.7789 ± 0.0023	0.8415 ± 0.0391	0.8211 ± 0.0681	0.8049 ± 0.0602
		LO	0.7063 ± 0.0201	0.9505 ± 0.0036	0.7902 ± 0.0034	**0.8528 ± 0.0588**	**0.8895 ± 0.0820**	**0.8051 ± 0.0618**
	GCN	OR	0.8630 ± 0.0219	0.8635 ± 0.0083	0.6707 ± 0.0040	0.3736 ± 0.0672	0.4579 ± 0.0711	0.4308 ± 0.0664
		FT	0.8683 ± 0.0191	0.9289 ± 0.0069	0.7624 ± 0.0051	0.4415 ± 0.1152	0.5526 ± 0.0832	0.5282 ± 0.0644
		LO	0.8694 ± 0.0177	0.9396 ± 0.0030	0.7703 ± 0.0020	0.5057 ± 0.0430	0.5789 ± 0.0588	0.5231 ± 0.0292
	GraphSAGE	OR	0.8720 ± 0.0216	0.8849 ± 0.0026	0.6864 ± 0.0011	0.6113 ± 0.0662	0.5053 ± 0.0776	0.6051 ± 0.0389
		FT	0.8592 ± 0.0363	0.9472 ± 0.0026	0.7881 ± 0.0019	0.7211 ± 0.1324	0.7579 ± 0.1123	0.7179 ± 0.1189
		LO	**0.8727 ± 0.0219**	0.9511 ± 0.0036	**0.7941 ± 0.0029**	0.7434 ± 0.0930	0.7737 ± 0.1311	0.6872 ± 0.0896
	MixHop	OR	0.8601 ± 0.0281	0.8969 ± 0.0038	0.6774 ± 0.0029	0.5736 ± 0.1183	0.5526 ± 0.1500	0.4974 ± 0.0803
		FT	0.8572 ± 0.0123	0.9493 ± 0.0030	0.7775 ± 0.0036	0.7092 ± 0.1035	0.7421 ± 0.1075	0.6718 ± 0.1397
		LO	0.8624 ± 0.0253	**0.9513 ± 0.0038**	0.7818 ± 0.0040	0.7094 ± 0.0738	0.8158 ± 0.0930	0.7179 ± 0.0314
Advanced GNNs	JK-Net		0.8579 ± 0.0001	0.8841 ± 0.0001	0.7532 ± 0.0012	0.7431 ± 0.0041	0.6649 ± 0.0046	0.6459 ± 0.0075
	H2GCN		0.8692 ± 0.0002	0.8940 ± 0.0001	0.7382 ± 0.0011	**0.8667 ± 0.0022**	**0.8486 ± 0.0044**	**0.8216 ± 0.0023**
	APPNP		0.8539 ± 0.0477	**0.9355 ± 0.0060**	0.7969 ± 0.0143	0.6830 ± 0.0470	0.7368 ± 0.0832	0.6410 ± 0.0480
	GCNII		**0.8833 ± 0.0027**	0.7925 ± 0.0043	0.7847 ± 0.0068	0.7020 ± 0.0037	0.7135 ± 0.0039	0.7405 ± 0.0060
	SGC		0.8509 ± 0.0648	0.8832 ± 0.0055	0.7740 ± 0.0160	0.5321 ± 0.0506	0.5526 ± 0.0811	0.4615 ± 0.0748
	SSP		0.8616 ± 0.0289	0.9178 ± 0.0116	**0.7976± 0.0185**	0.6302 ± 0.0850	0.7000 ± 0.0758	0.6923 ± 0.0314

Main Results / Performance Comparison (RQ1)

RQ1: Does the LOGIN framework achieve performance comparable to state-of-the-art GNNs?

- Classic GNNs + LOGIN outperform vanilla baselines
- Achieves performance on par with advanced GNNs
- Best results on PubMed (homophilic) and Texas (heterophilic)

Ablation Study (RQ2)

RQ2: How does the complementary coping mechanism for LLMs' responses contribute to the LOGIN framework?

- Complete LOGIN framework consistently outperforms all ablated variants
- Homophilic graphs -> Feature update more effective
- Heterophilic graphs -> Structure refinement crucial

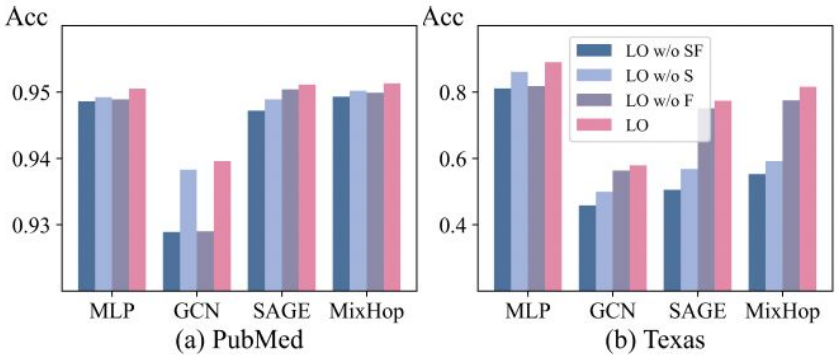


Figure 3: Ablation studies of LOGIN on PubMed and Texas.

Case Study (RQ3)

RQ3: How does LOGIN operate over specific nodes based on responses from LLMs?

- Case 1: Cora (Correct)
  - LLM: Correctly classified as 'Neural Networks' with rationale
  - Action: Feature enhancement
- Case 2: Wisconsin (Wrong)
  - LLM: Misclassified as 'student' (should be 'course')
  - Action: Edge pruning

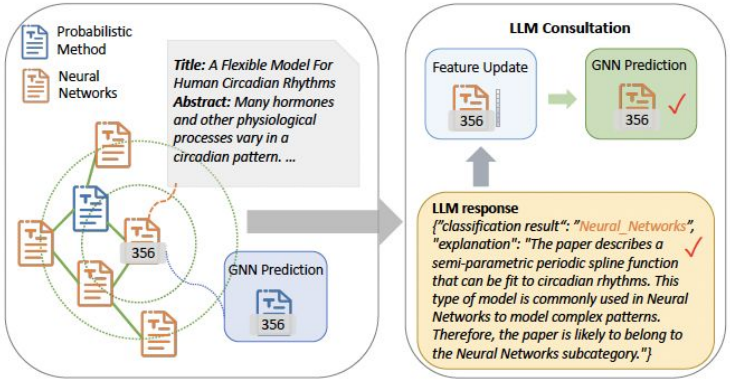


Figure 4: Case study of node 356 in Cora, where the LLM consultation classified right and performed feature update.

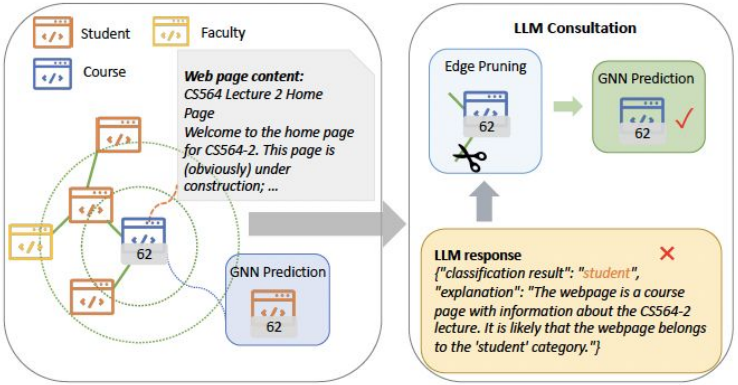


Figure 5: Case study of node 62 in Wisconsin, where the LLM consultation misclassified and pruned edges.

Comparison among LLM-based Paradigms (RQ4)

RQ4: How does LLMs-as-Consultants paradigm perform compared with LLMs-as-Predictors and LLMs-as-Enhancers?

Table 2: Performance comparison among LLMs-as-Predictors, LLMs-as-Enhancers and LLMs-as-Consultants paradigms.

Dataset	Method	LLMs-as-Predictors (vicuna-v1.5-7b)	LLMs-as-Enhancers (TAPE + llama2-13b-chat)	LLMs-as-Consultants (LOGIN + vicuna-v1.5-7b)
Cora	MLP	0.7432 ± 0.0131	0.7675 ± 0.0187	0.7343 ± 0.0841
	GCN		0.8630 ± 0.0101	**0.8759 ± 0.0151**
	GraphSAGE		0.8625 ± 0.0093	0.8699 ± 0.0167
PubMed	MLP	0.7847 ± 0.0632	0.9475 ± 0.0046	**0.9508 ± 0.0040**
	GCN		0.9257 ± 0.0063	0.9401 ± 0.0043
	GraphSAGE		0.9464 ± 0.0033	0.9505 ± 0.0031
Arxiv-23	MLP	0.7547 ± 0.0927	0.7905 ± 0.0041	0.7909 ± 0.0064
	GCN		0.7751 ± 0.0029	0.7734 ± 0.0028
	GraphSAGE		0.7935 ± 0.0029	**0.7961 ± 0.0029**

Comparison among LLM-based Paradigms (RQ4)

RQ4: How does LLMs-as-Consultants paradigm perform compared with LLMs-as-Predictors and LLMs-as-Enhancers?

- LLMs-as-Consultants Advantages:
  - Better performance with smaller LLM
  - Consults only subset of nodes (lower cost)
  - Interactive integration during training
  - Lower time and resource consumption

Extensive Study (RQ5)

RQ5: Can consulting more advanced LLMs, consulting with more nodes, or applying more advanced GNNs in LOGIN unlock greater potential?

- More Advanced LLMs
  - Vicuna-v1.5-13b and GPT 3.5-turbo-0125: Progressive accuracy improvements (Cora: 70.63% → 72.02% → 81.23%)
- Consulting More Nodes & Advanced GNN Backbones
  - Increasing nodes in consultation by 10% yields notable performance gains
  - LOGIN enhances performance of advanced GNNs (APPNP, GCNII) beyond classic ones

Table 3: Performance comparison among LOGIN with different LLMs on Cora.

LLMs	MLP	GCN	GraphSAGE
Vicuna-v1.5-7b	0.7063 ± 0.0331	0.8694 ± 0.0102	0.8727 ± 0.0201
Vicuna-v1.5-13b	0.7202 ± 0.0201	0.8702 ± 0.0191	0.8739 ± 0.0135
GPT 3.5-turbo-0125	0.8123 ± 0.0254	0.8992 ± 0.0099	0.8856 ± 0.0102

Table 5: Performance comparison of consulting with more nodes on Cora and Wisconsin.

Method	Cora	Wisconsin
MLP+LOGIN	0.7063 ± 0.0331	0.8528 ± 0.0588
MLP+LOGIN+10% nodes	0.7147 ± 0.0318	0.8605 ± 0.0493
GrraphSAGE+LOGIN	0.8727 ± 0.0219	0.7434 ± 0.0930
GraphSAGE+LOGIN+10% nodes	0.8789 ± 0.0310	0.7527 ± 0.0698

Table 6: Performance comparison of APPNP and GCNII trained with or without the LOGIN framework.

Method	Cora	Wisconsin
APPNP	0.8539 ± 0.0477	0.7969 ± 0.0143
APPNP+LOGIN	0.8614 ± 0.0332	0.7973 ± 0.0202
GCNII	0.8833 ± 0.0027	0.7847 ± 0.0068
GCNII+LOGIN	0.8879 ± 0.0105	0.7903 ± 0.0121

Key Contributions

- New “LLMs-as-Consultants” Paradigm
  - First to propose LLMs-as-Consultants for graph machine learning with interactive integration
- LOGIN Framework
  - Complementary feedback strategy utilizing both correct and incorrect LLM responses
- Strong Empirical Results
  - Classic GNNs + LOGIN match/exceed advanced GNNs across both homophilic and heterophilic graphs

Limitations & Future Works

Liminations

- Requires text-attributed graphs (TAGs)
- Node text might be sparse, noisy, and expensive to obtain
  - LLMs guess wrong -> doesn’t always mean topological noise, could be semantic ambiguity
  - May lead to unnecessary pruning

Future Works

- reduce the reliance on raw node text
  - Generate text features from other available attributes
- Node Selection
  - Check neighbourhood-level uncertainty
- Refinement
  - Add LLM confidence score
- Prompt
  - Topology context (1-hop or 2-hop), etc

Conclusion

- New paradigm of leveraging LLMs for graph tasks: “LLMs-as-Consultants”
- LOGIN Framework empowers interactive LLM consultation in the GNN training process
  - Semantic + topology upgrade